E4 — ARQUITETURA, MODELAGEM E BACKLOG

Eduardo Krause (RA 12520157) — Análise e Desenvolvimento de Sistemas — UNIMATER
Orientador: Prof. Me. João Mazzochin — Entrega de 13/10/2026

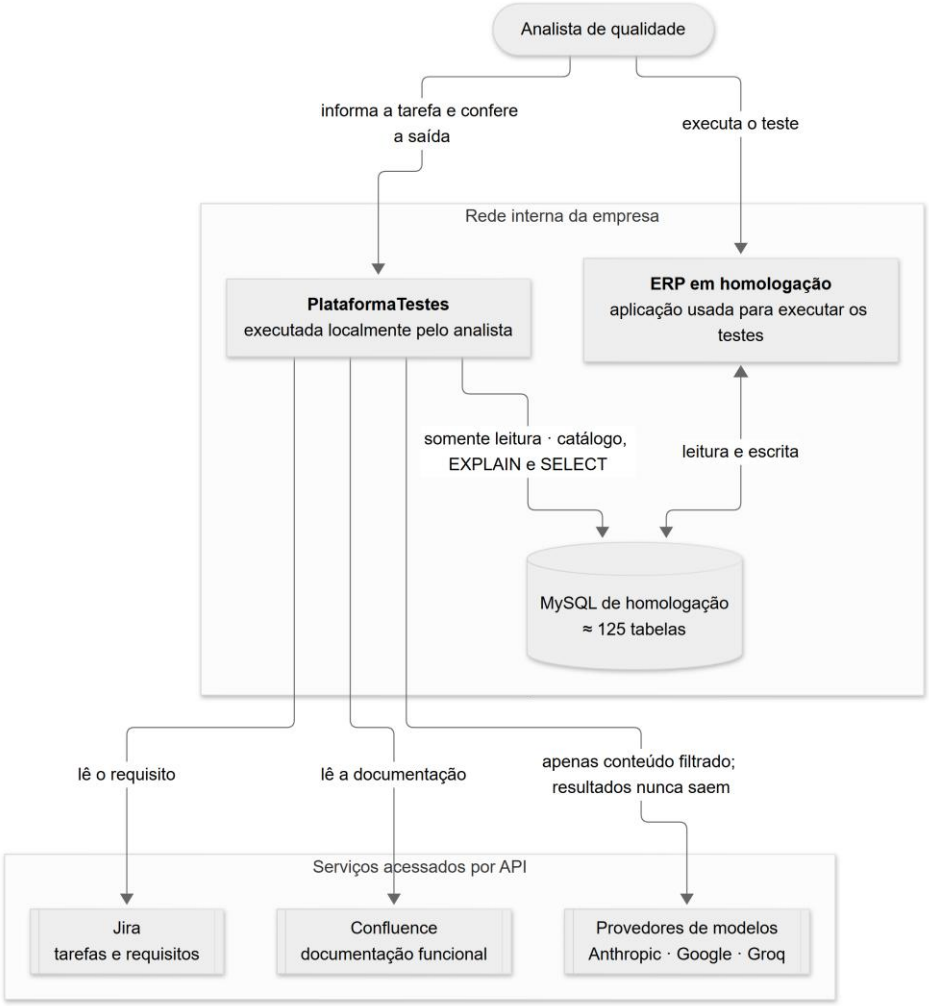
Este documento leva a proposta do papel para o projeto executável: contexto em relação ao ERP, arquitetura, fluxo principal, dados enviados aos provedores de modelos, modelagem dos dados da plataforma, stack justificada e backlog priorizado com a fatia vertical marcada. As seções 1 e 4 atendem a pontos da avaliação da proposta. Os diagramas foram escritos em Mermaid e estão versionados no repositório do projeto, com os mesmos nomes de componentes usados aqui e no código. O conteúdo alimenta as subseções 3.1 a 3.3 do artigo.

1 Contexto: a plataforma e o ERP

O sistema estudado é um ERP web, acessado pelo navegador, com banco de dados MySQL de cerca de 125 tabelas. A avaliação usa o ambiente de homologação, no qual os analistas executam os testes antes de cada entrega semanal. Por confidencialidade, o sistema não é identificado pelo nome.

A Figura 1 mostra onde a plataforma se conecta. Ela não se integra à aplicação do ERP, nem à sua interface ou às suas regras de negócio: lê o banco de homologação em modo somente leitura, o requisito no Jira e a documentação no Confluence. Esse acoplamento restrito ao catálogo e aos dados é o que permite à plataforma operar sobre qualquer área funcional, como afirma a proposta, e a torna independente da tecnologia em que o ERP foi construído.

Figura 1 – Contexto da plataforma em relação ao ERP e aos serviços que consulta

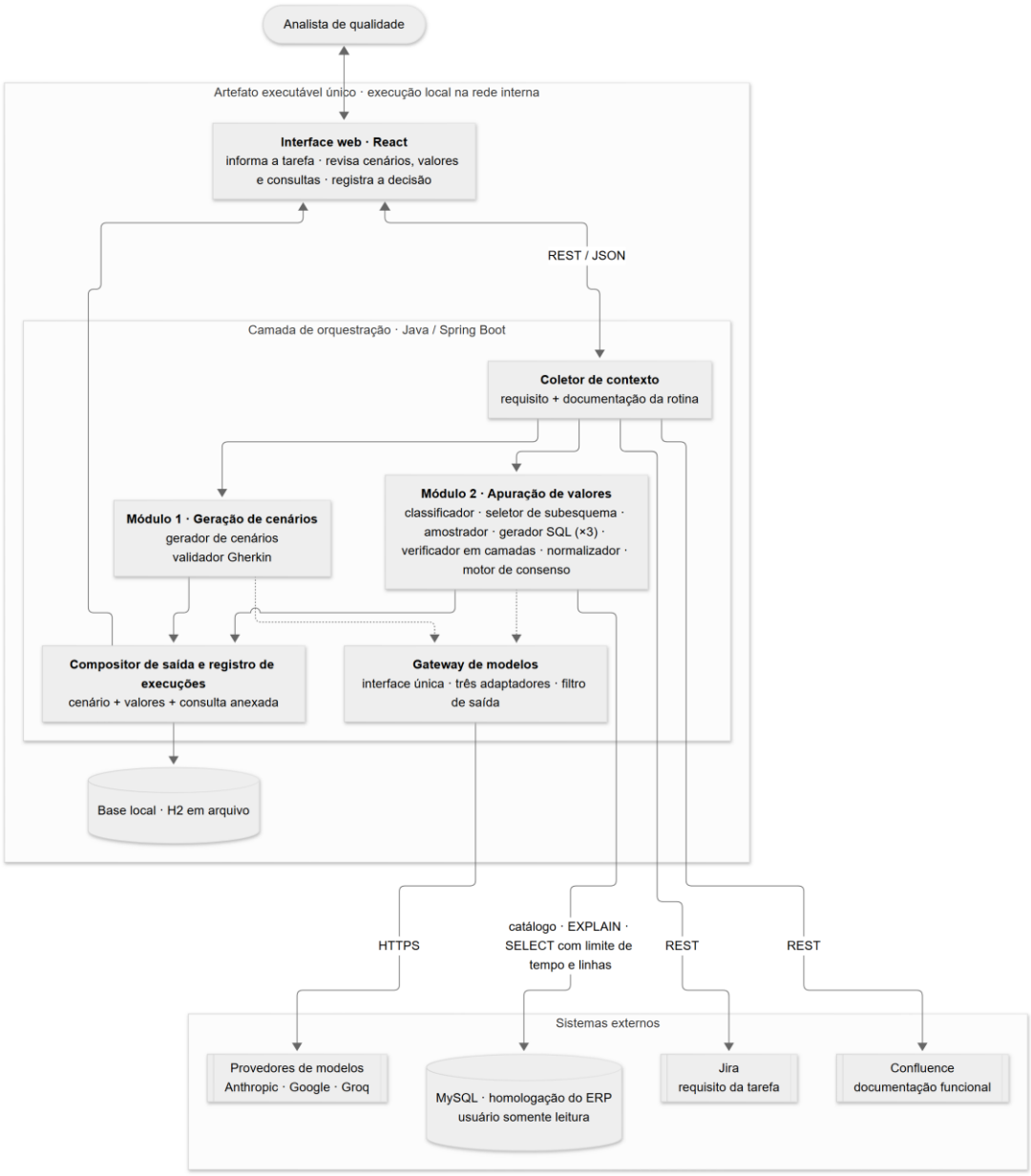


Fonte: elaborado pelo autor (2026).

2 Arquitetura da solução

A plataforma é um monólito modular empacotado em um único executável e executado localmente na rede interna, como definido na proposta. A escolha descarta deliberadamente uma arquitetura distribuída: trata-se de um projeto de uma pessoa, com prazo de um semestre, e a única exigência que pediria separação física, manter credenciais e banco fora do navegador, é atendida pela própria execução local. A Figura 2 apresenta os componentes e a forma como se comunicam.

Figura 2 – Arquitetura da plataforma



Fonte: elaborado pelo autor (2026).

A **interface web** é o único ponto de contato do analista: recebe a chave da tarefa, apresenta os cenários com os valores apurados e as consultas que os originaram, e registra a decisão de conferência. Comunica-se com a camada de orquestração por REST/JSON, servida pelo mesmo processo.

Na **camada de orquestração**, o **coletor de contexto** obtém o requisito no Jira e a documentação da rotina no Confluence, e entrega esse contexto aos dois módulos. O

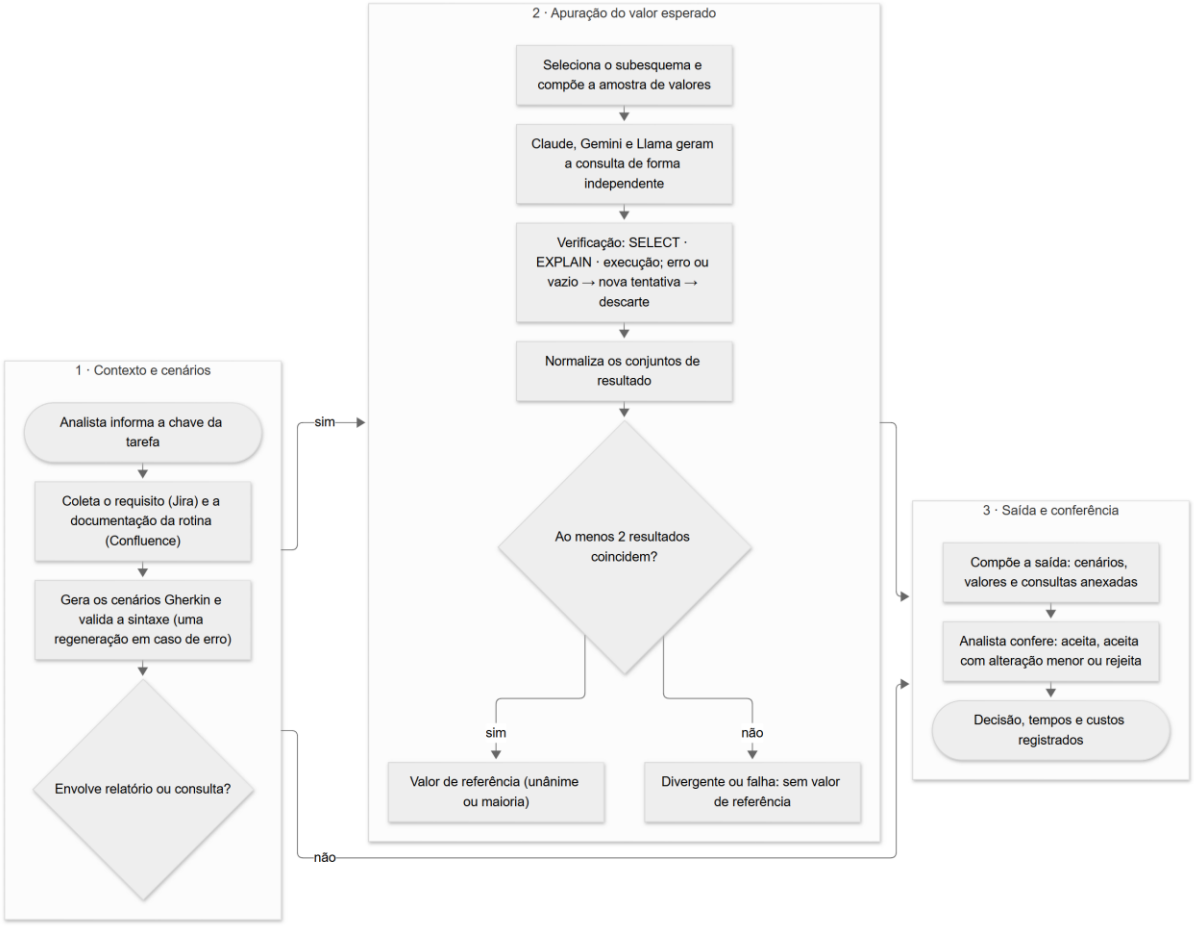
Módulo 1 redige os cenários e os submete ao validador Gherkin. O **Módulo 2** é acionado apenas quando o requisito envolve relatório ou consulta e encadeia as etapas descritas na proposta, da classificação do requisito ao motor de consenso. Ambos os módulos acessam os modelos exclusivamente pelo **gateway de modelos**, que oferece uma interface única com um adaptador por provedor; com isso, trocar a versão ou o provedor de um modelo não altera os módulos, e tokens, custo e latência são registrados em um único ponto. O gateway abriga também o filtro de saída descrito na seção 4, de modo que nenhum conteúdo chega a um provedor sem passar por ele. O **compositor de saída** reúne cenário, valores e consultas, e o **registro de execuções** grava tudo na base local, que alimenta a auditoria e as métricas da avaliação.

A base de homologação do ERP é acessada apenas pelo Módulo 2, com usuário de permissão exclusiva de leitura e limites de tempo e de linhas em toda consulta. A plataforma nunca escreve nessa base: seus próprios dados ficam na base local, separada.

3 Fluxo principal

A Figura 3 mostra o caminho de uma tarefa da entrada à conferência, em três etapas. A etapa 2 só é percorrida quando o classificador conclui que o requisito exige apuração de valores no banco.

Figura 3 – Fluxo principal de uma tarefa



Fonte: elaborado pelo autor (2026).

Cada candidato a consulta atravessa três camadas sucessivas: a análise sintática, que admite uma única instrução do tipo SELECT; o plano de execução, obtido por EXPLAIN; e a execução propriamente dita. Um candidato com erro ou com retorno vazio recebe uma nova tentativa, na qual a mensagem de erro é devolvida ao modelo; persistindo a falha, é descartado. Os candidatos válidos têm o conjunto de resultado normalizado segundo regras explícitas, para que diferenças de forma não sejam confundidas com diferenças de conteúdo:

– nomes e ordem das colunas são desconsiderados, pois cada modelo escolhe seus próprios apelidos;
– a ordem das linhas é desconsiderada, salvo quando o requisito especifica ordenação;

– valores numéricos são arredondados à escala da coluna de origem ou, na ausência dela, a duas casas;
– nulos recebem marcador próprio, distinto de texto vazio, e datas são escritas no formato ISO 8601;
– o conjunto normalizado é resumido por um *hash*, e dois candidatos concordam quando os *hashes* coincidem.

A decisão de consenso classifica cada apuração em um de quatro desfechos, descritos no Quadro 1.

Quadro 1 – Desfechos da decisão por consenso

Desfecho	Condição	Saída entregue ao analista
Unânime	Três candidatos válidos com o mesmo resultado	Valor de referência, com indicação de confiança alta
Maioria	Dois resultados coincidentes; o terceiro diverge ou foi descartado	Valor de referência, com indicação de confiança moderada e a consulta divergente exibida
Divergente	Dois ou mais candidatos válidos, sem resultados coincidentes	Nenhum valor de referência; as consultas são exibidas para apuração pelo analista
Falha	Menos de dois candidatos válidos após a nova tentativa	Nenhum valor de referência; os erros de cada modelo são registrados

Fonte: elaborado pelo autor (2026).

Em todos os desfechos a conferência humana é obrigatória. A plataforma entrega um valor **apurado**, obtido do banco, rastreável e reprodutível; ele só se torna **validado**, isto é, representativo da regra de negócio, depois da conferência do analista. O consenso não aprova o valor: indica o grau de confiança associado a ele, e a consulta anexada permite verificar sua origem sem reconstruí-la.

4 Dados enviados aos provedores e minimização

O acesso ao banco é somente leitura, mas parte do conteúdo da empresa precisa sair dela para que os modelos funcionem. O Quadro 2 inventaria o que é enviado em cada etapa e a medida aplicada. O princípio é enviar o mínimo necessário à tarefa e nunca enviar resultados de consultas: a normalização e o consenso ocorrem inteiramente dentro da plataforma.

Quadro 2 – Inventário do conteúdo enviado aos provedores de modelos

Etapa	O que é enviado	O que pode conter	Medida aplicada
Geração de cenários e classificação	Texto do requisito e da documentação da rotina	Nomes de clientes, documentos, e-mails ou telefones usados como	Pseudonimização pelo filtro de saída; documentação restrita à página da rotina

Etapa	O que é enviado	O que pode conter	Medida aplicada
		exemplo; informação interna	
Seleção de esquema e geração de SQL	Nomes de tabelas, colunas, tipos e chaves do subconjunto pertinente	Metadados confidenciais, sem dados pessoais	Envio apenas do subesquema; coberto pela autorização da empresa
Composição de amostra	Valores distintos de colunas de baixa cardinalidade e limites de datas	Dados reais da base	Somente colunas da lista de permitidas; texto livre nunca é amostrado
Nova tentativa	Consulta anterior e mensagem de erro do banco	Valores citados na mensagem de erro	Mensagem passa pelo filtro de saída
Normalização e consenso	Nada	—	Resultados das consultas nunca saem da plataforma

Fonte: elaborado pelo autor (2026).

As medidas combinam três técnicas, aplicadas em pontos definidos do fluxo:

– **lista de colunas permitidas**, no amostrador: só são amostradas colunas explicitamente aprovadas, como situação, tipo de movimento e unidade, revisadas com o responsável pela base; uma coluna nova fica de fora até ser avaliada, o que é mais seguro do que uma lista de bloqueio;

– **pseudonimização**, no filtro de saída do gateway: CPF, CNPJ, e-mail, telefone e nomes de pessoas cadastradas são substituídos por marcadores consistentes, como `<CPF_1>`, antes do envio, e a tabela de correspondência permanece apenas na base local;

– **supressão**: resultados de consultas, valores fora da lista de permitidas e campos de texto livre nunca são enviados.

Cada envio é registrado na base local com o conteúdo efetivamente transmitido, o que permite auditar o que saiu da empresa. A configuração adotada em cada provedor, conferida nos termos vigentes em 12/10/2026, é declarada no Quadro 3.

Quadro 3 – Configuração adotada em cada provedor

Provedor	Modalidade adotada	Uso dos dados para treinamento	Retenção declarada
Anthropic (Claude)	API comercial, com chave da organização	Não usa entradas e saídas para treinar modelos, salvo permissão expressa	Exclusão automática em até 30 dias; até 2 anos em caso de violação da política de uso
Google (Gemini)	API em nível pago, com faturamento ativo; o nível gratuito não é usado	No nível pago, não usa as entradas e saídas para melhorar produtos; no gratuito, usa e admite revisão humana	Registro por período limitado, apenas para detectar abuso; registro opcional de chamadas mantido desativado
Groq (Llama)	API do GroqCloud, com retenção zero ativada	Vedado pelo contrato, salvo permissão do cliente	Nenhuma com retenção zero; sem ela, até 30 dias para confiabilidade e abuso

Fonte: elaborado pelo autor com base em Anthropic (2026), Google (2026) e Groq (2026).

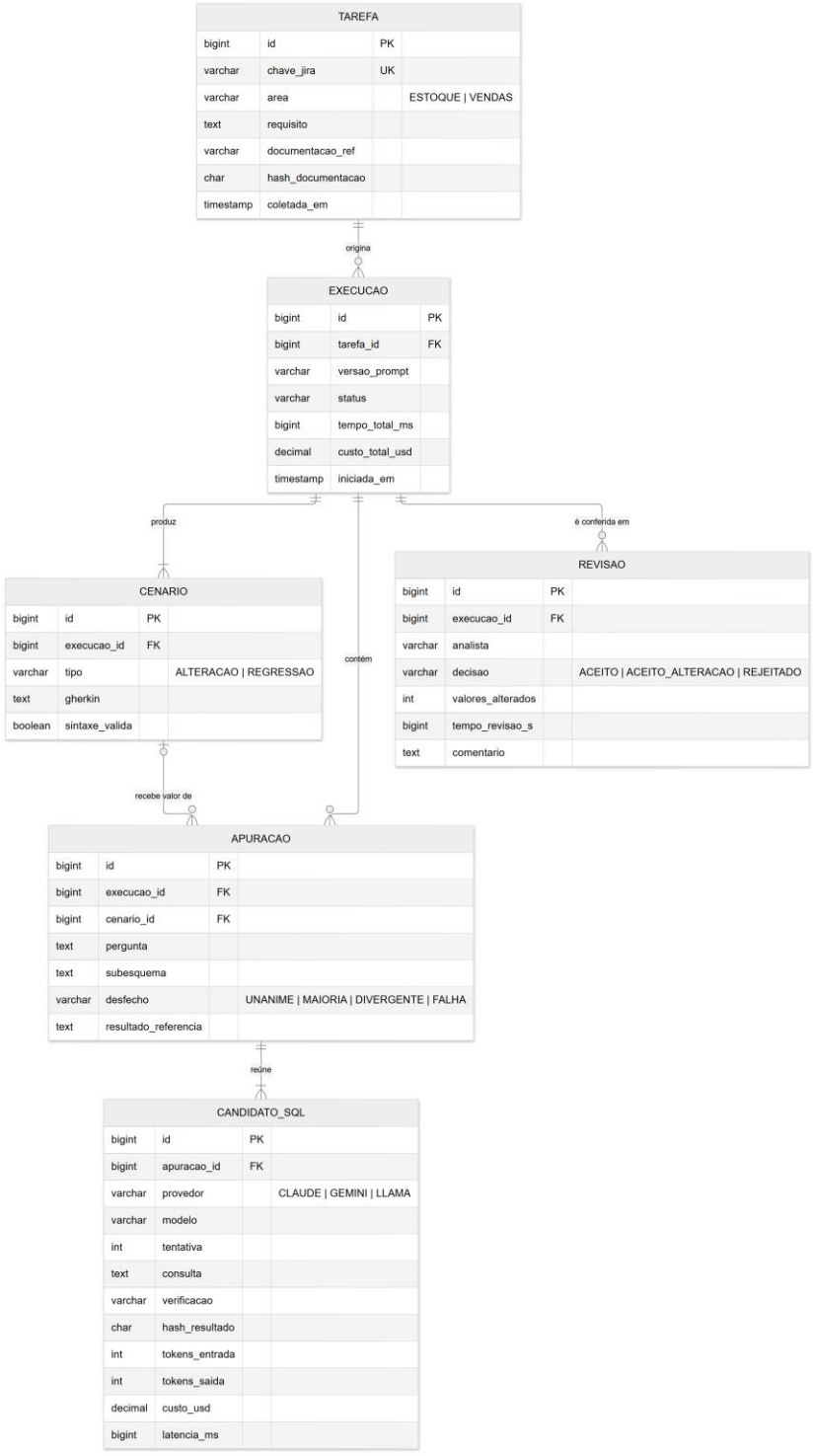
Duas decisões decorrem do quadro. O nível gratuito do Gemini fica vedado, porque nele o conteúdo pode ser lido por revisores humanos e usado para melhorar produtos; e a retenção zero é ativada no Groq, único provedor que a oferece ao próprio cliente. Nos demais casos, a retenção de até 30 dias recai sobre conteúdo já minimizado e pseudonimizado.

A empresa autorizou o uso do ambiente de homologação, das tarefas e da documentação, com a condição de que nenhuma informação sensível seja exposta. As medidas desta seção são a forma de cumprir essa condição.

5 Modelagem de dados

O esquema do ERP, com cerca de 125 tabelas, é apenas lido e não faz parte da modelagem. O que se modela são os dados da própria plataforma, restritos ao necessário para auditar cada execução e calcular as métricas da avaliação. A Figura 4 apresenta as seis entidades.

Figura 4 – Modelo entidade-relacionamento da base local



Fonte: elaborado pelo autor (2026).

Quadro 4 – Entidades da base local e métricas que alimentam

Entidade	**Papel**	**Métricas que alimenta**
TAREFA	Retrato do requisito e da referência à	Identificação do conjunto de

Entidade	Papel	Métricas que alimenta
	documentação no momento da coleta	avaliação
EXECUCAO	Uma passagem da plataforma sobre uma tarefa, com versão do prompt, tempo e custo totais	Custo e tempo por tarefa
CENARIO	Cada cenário gerado, marcado como de alteração ou de regressão	Cobertura e completude
APURACAO	Cada valor a apurar, com o subesquema usado, o desfecho e o resultado de referência	Concordância, acurácia, falso acordo e divergência
CANDIDATO_SQL	Cada consulta gerada por um modelo, com a verificação, o *hash* do resultado, os tokens e a latência	Acurácia por modelo, custo e tempo por consulta
REVISAO	A conferência do analista: decisão, valores alterados e tempo gasto	Aceitação e proporção de valores alterados

Fonte: elaborado pelo autor (2026).

A coluna `hash_documentacao` em TAREFA registra qual versão da documentação foi usada, já que as páginas do Confluence mudam com o tempo; sem ela, um experimento não poderia ser repetido com a mesma entrada.

6 Stack definida e justificada

Quadro 5 – Stack da plataforma

Tecnologia	Função e justificativa
Java 21 e Spring Boot 3	Camada de orquestração. Acesso maduro a bancos relacionais por JDBC, execução concorrente das três chamadas aos modelos e empacotamento em um único JAR executável. Java 21 é a versão de suporte estendido corrente.
Spring AI	Cliente dos modelos. Oferece abstração comum para Anthropic, Google e provedores compatíveis com a interface da OpenAI, caso da Groq, o que reduz cada adaptador do gateway a configuração. Se alguma limitação surgir, o adaptador afetado passa a usar o cliente HTTP do próprio provedor, sem impacto nos módulos.
React com TypeScript e Vite	Interface. O resultado do *build* é copiado para os recursos estáticos do Spring Boot e servido pelo mesmo processo, preservando o artefato único.
H2 em arquivo	Base local da plataforma. Não exige servidor, roda dentro do processo e mantém os dados da plataforma fisicamente separados da base do ERP.
MySQL Connector/J	Acesso à homologação com usuário somente leitura. O limite de tempo é aplicado pela dica `MAX_EXECUTION_TIME` e o de linhas pelo próprio *driver*.
JSqlParser	Primeira camada do verificador. Rejeita qualquer texto que não seja uma única instrução SELECT, inclusive múltiplas instruções e cláusulas de bloqueio, antes que a consulta chegue ao banco.
Analisador Gherkin do Cucumber	Validador do Módulo 1. É o mesmo analisador usado pelas ferramentas de BDD, o que torna a métrica de validade sintática comparável à reportada na literatura.
APIs REST do Jira e do Confluence	Coleta de contexto, com *token* de acesso do autor e apenas operações de leitura no MVP.
Git, GitHub e Mermaid	Versionamento do código e dos diagramas como texto. Os diagramas deste documento são gerados a partir dos arquivos .mmd do repositório.

Fonte: elaborado pelo autor (2026).

As versões exatas dos três modelos serão fixadas antes dos experimentos e registradas em cada execução, na coluna `modelo` de CANDIDATO_SQL, para que mudanças de versão pelo provedor não contaminem a comparação.

7 Backlog priorizado

O Quadro 6 lista os itens em ordem de prioridade. Nenhum item passa de oito horas, e cada um está associado ao marco em que deve estar pronto. Os itens da fatia vertical estão marcados com ●. O MVP soma 125 horas e os itens desejáveis, 22 horas.

Quadro 6 – Backlog priorizado

ID	Item	h	Prioridade	Marco	FV
B01	Esqueleto Spring Boot + React no mesmo build, gerando um único executável	4	MVP	E6	●
B02	Configuração de credenciais por variáveis de ambiente e perfil local	1	MVP	E6	●
B03	Base local H2 com migração das seis tabelas da plataforma	3	MVP	E6	●
B04	Cliente do Jira: obter o requisito pela chave da tarefa	3	MVP	E6	●
B05	Gateway de modelos com adaptador do Claude e registro de tokens, custo e latência	4	MVP	E6	●
B06	Conexão MySQL somente leitura com limite de tempo e de linhas	2	MVP	E6	●
B07	Verificador, camada 1: análise sintática que admite apenas SELECT	3	MVP	E6	●
B08	Geração de cenário Gherkin a partir do requisito (um modelo)	4	MVP	E6	●
B09	Geração de SQL por um modelo, com o esquema da área informado integralmente	4	MVP	E6	●
B10	Compositor: cenário com valor preenchido e consulta anexada	2	MVP	E6	●
B11	Tela: informar a tarefa e visualizar o resultado	4	MVP	E6	●
B12	Adaptadores do Gemini e do Llama (Groq) no gateway	4	MVP	E7	
B13	Geração paralela pelos três modelos e nova tentativa com a mensagem de erro	6	MVP	E7	
B14	Verificador, camadas 2 e 3: EXPLAIN e execução; descarte de erro e de vazio	4	MVP	E7	
B15	Normalizador: ordenação, arredondamento, nulos e datas	6	MVP	E7	
B16	Motor de consenso e classificação do desfecho	4	MVP	E7	
B17	Extração do esquema pelo INFORMATION_SCHEMA e grafo de chaves estrangeiras	6	MVP	E7	
B18	Amostrador de valores restrito à lista de colunas permitidas	5	MVP	E7	
B19	Filtro de saída no gateway: pseudonimização de dados pessoais no requisito, na documentação e nas mensagens de erro	5	MVP	E7	
B20	Cliente do Confluence: obter a página da rotina e convertê-la em texto	6	MVP	E8	
B21	Documentação funcional no prompt, com cenários de regressão	6	MVP	E8	
B22	Validação com o analisador Gherkin e uma regeneração em caso de erro	4	MVP	E8	
B23	Classificador: o requisito exige apuração de valores?	4	MVP	E8	
B24	Restrições de domínio obrigatórias no prompt (empresa, filial, cancelados, exclusão lógica)	4	MVP	E8	

ID	Item	h	Prioridade	Marco	FV
B25	Seleção do subesquema por similaridade de termos e expansão pelos relacionamentos	8	MVP	TCC II	
B26	Tela de revisão: aceitar, aceitar com alteração ou rejeitar, com registro de tempo	6	MVP	TCC II	
B27	Exibição do desfecho e da consulta de cada modelo	4	MVP	TCC II	
B28	Execução em lote sobre o conjunto de referência	6	MVP	TCC II	
B29	Exportação das métricas em CSV	3	MVP	TCC II	
B30	Registro dos cenários aprovados de volta na tarefa do Jira	6	Desejável	—	
B31	Cache do esquema e das amostras entre execuções	4	Desejável	—	
B32	Painel de custos por provedor	4	Desejável	—	
B33	Incorporação das alterações de código vinculadas à tarefa	8	Desejável	Futuro	

Fonte: elaborado pelo autor (2026).

7.1 Fatia vertical

A fatia vertical, itens B01 a B11, é o menor caminho que atravessa todas as camadas e prova a ideia central: o analista informa a chave de uma tarefa de relatório; a plataforma lê o requisito no Jira, gera o cenário e a consulta com um único modelo, recebendo o esquema da área integralmente, aceita apenas SELECT, executa a consulta em modo somente leitura e devolve o cenário com o valor preenchido e a consulta anexada, gravando a execução na base local. Soma 34 horas e é o conteúdo da E6. O consenso entre os três modelos, que é o diferencial do trabalho, entra logo em seguida, na E7, sobre essa mesma fatia.

7.2 Capacidade e risco de prazo

Até a banca de validação, os marcos E6, E7 e E8 somam 98 horas de implementação (34, 40 e 24 horas, respectivamente), acima das cerca de 74 horas previstas no cronograma das entregas para o mesmo período. A ordem do backlog foi construída para que o excesso recaia sobre o final: se for preciso cortar, a banca recebe a fatia vertical com consenso funcionando e o Módulo 1 sem a documentação funcional, e os itens restantes passam ao início do TCC II, junto das 27 horas já previstos ali para preparar o experimento. A seleção do subesquema (B25) foi deixada para o TCC II porque a fatia funciona enviando o esquema inteiro da área, o que a proposta admite como viável; o custo dessa simplificação será medido.

8 Repositório

O repositório foi criado com README descrevendo o projeto, os diagramas em Mermaid e o backlog em Markdown, gerado da mesma fonte que o Quadro 6. Ainda não há código: o primeiro *commit* de implementação será o esqueleto do item B01.

```
PlataformaTestes/
├── README.md
├── .gitignore
└── docs/
    ├── backlog.md
    └── diagramas/
        ├── contexto.mmd      contexto.svg      contexto.png
        ├── arquitetura.mmd   arquitetura.svg   arquitetura.png
        ├── fluxo.mmd         fluxo.svg         fluxo.png
        └── der.mmd           der.svg           der.png
```

Endereço: https://github.com/edukrause/PlataformaTestes

Referências

ANTHROPIC. **How long do you store my organization's data?** 2026. Disponível em: https://privacy.claude.com/en/articles/7996866-how-long-do-you-store-my-organization-s-data. Acesso em: 12 out. 2026.

ANTHROPIC. **Is my data used for model training?** 2026. Disponível em: https://privacy.claude.com/en/articles/7996868-is-my-data-used-for-model-training. Acesso em: 12 out. 2026.

GOOGLE. **Gemini API Additional Terms of Service**. 2026. Disponível em: https://ai.google.dev/gemini-api/terms. Acesso em: 12 out. 2026.

GROQ. **Your Data in GroqCloud**. 2026. Disponível em: https://console.groq.com/docs/your-data. Acesso em: 12 out. 2026.

Observações sobre esta entrega

1. **Ajustes da avaliação da proposta.** A seção 1 atende ao pedido de Douglas Bianchezzi sobre a estrutura do ERP, e a seção 4 aos de Rafael Rodrigues Barbosa e Anderson Luiz Fernandes sobre minimização e anonimização dos dados. O item B19 do backlog implementa o filtro de saída.
2. **Autorização da empresa.** Concedida pela gestão, com a condição de não expor informação sensível, atendida pelas medidas da seção 4.
3. **Prazo.** Esta entrega e a E3 foram enviadas depois das datas previstas. O backlog e a seção 7.2 já consideram o calendário real até a banca.